PARTICIPANT HANDOUT / PRACTICE SET 02

PySpark Hands-on Exercises

Logistics & Delivery Operations

26 exercises + final challenge | Platform: Google Colab / PySpark

Dataset files

File	Records	Purpose
shipments_75000.csv	75,000	Main shipment transactions
shipments_extra_12000.csv	12,000	Additional batch; includes 750 repeated shipment IDs
warehouse_master.csv	8	Warehouse and region reference
carrier_master.csv	7	Carrier reference and reliability
route_master.csv	12	Route distances and tolls
warehouse_sensor_config.json	8	Nested warehouse sensor configuration

Instructions: Upload the six files to Google Colab, create PySpark DataFrames, and complete each task independently. Use transformations and actions appropriately. Include code and output in your notebook. No solutions are included.

PART A | DataFrame Operations

01 Display the schema, total number of records, and first 15 shipments.

02 Select shipment_id, warehouse_id, service_level, shipment_status, and shipping_fee_inr.

03 Filter shipments where service_level = "Express" and package_weight_kg > 5.

04 Sort shipments by shipping_fee_inr in descending order and display the top 20.

PART B | Aggregations and Joins

05 Calculate the total number of shipments for each shipment_status.

06 Calculate total shipping fees, total shipping costs, and profit margin for each service_level. Handle missing cost values appropriately.

07 Join shipments with the warehouse master and calculate the number of shipments originating from each region.

08 Join shipments, carriers, and routes. Calculate average shipping cost per kilometre for each carrier, excluding missing costs and zero distances.

PART C | Null Handling and Conditional Logic

09 Count missing values in carrier_id, package_weight_kg, shipping_cost_inr, and customer_rating.

10 Replace null handling_minutes values with zero in one DataFrame. Create another DataFrame that excludes shipments with a missing carrier_id.

11 Create weight_category using when() and otherwise(): Light (< 2 kg), Medium (2-10 kg), Heavy (> 10 kg), and Unknown (null).

PART D | Date and Time Functions

12 Convert delivery-related date columns to Spark date types and calculate delivery duration in days for delivered shipments.

13 Create delivery_performance as Early, On Time, or Late by comparing actual and promised delivery dates. Give missing actual dates a separate classification.

14 Using booking_date, calculate monthly delivered-shipment counts and total shipping fees.

PART E | Window Functions

15 Rank carriers by total shipping fees within each warehouse region using dense_rank().

16 Identify the latest booked shipment for each warehouse using row_number().

17 Calculate a running total of shipping fees for each warehouse, ordered by booking date and shipment ID.

PART F | Union and Complex JSON

18 Combine shipments_df and extra_df using unionByName(). Count the combined records.

19 Identify repeated shipment_id values and remove duplicate shipment records. Compare row counts before and after deduplication.

20 From the nested sensor JSON, extract warehouse ID, temperature limits, and sensor details. Use explode() to produce one row per sensor.

PART G | Lazy Execution, Caching, and Persistence

21 Build a pipeline using filter(), join(), withColumn(), groupBy(), and orderBy(). Inspect its plan with explain() before calling an action.

22 Cache delivered shipments. Compare execution times for repeated count() and aggregation actions.

23 Persist Express shipments using MEMORY_AND_DISK. Run two actions, inspect the storage level, and release the data using unpersist().

PART H | Partitioning and Optimization

24 Repartition shipments into 8 partitions. Use spark_partition_id() to count records in each partition.

25 Repartition shipments into 12 partitions, then reduce to 3 using coalesce(). Compare with repartition(3) using execution plans.

26 Join shipment and warehouse data, then write the result as Parquet using 4 output partitions. Inspect the generated files in Colab.

FINAL CHALLENGE | Logistics Performance Pipeline

Build one end-to-end PySpark pipeline that joins shipment, warehouse, carrier, and route data. Complete all requirements below:

1. Clean missing shipping costs and package weights using a clearly explained strategy.
2. Calculate delivery duration, late-delivery indicators, and shipment profit.
3. Aggregate revenue, profit, and late-delivery percentage by region and carrier.
4. Rank carriers within each region by on-time delivery percentage.
5. Cache the final aggregated DataFrame and reuse it for multiple actions.
6. Save the final report as partitioned Parquet files.

Submission: One Google Colab notebook with completed tasks, outputs, relevant execution plans, and the final logistics performance report.